

LLM (LARGE LANGUAGE MODEL) ARCHITECTURE

LLMs learn patterns in text and generate text based on those learned patterns from massive amounts of text data.

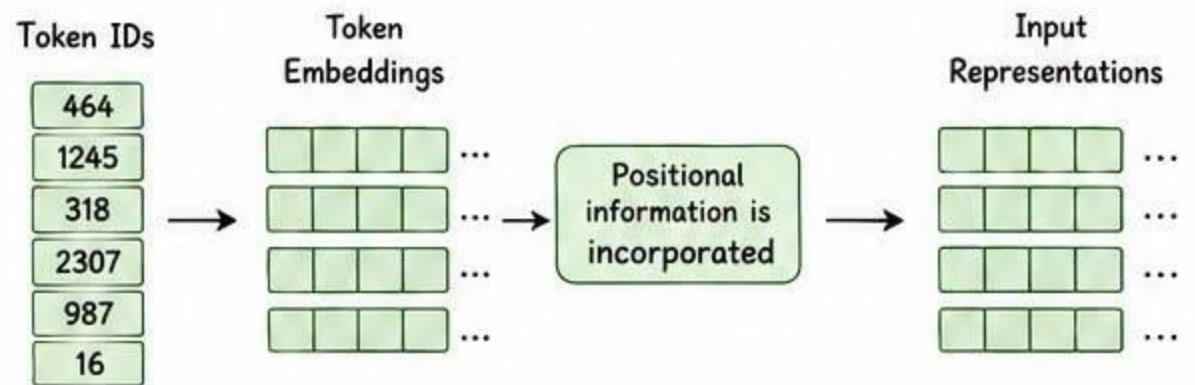
1. INPUT (TOKENIZATION)

- Text is broken into small units called tokens
- Tokens are converted into numeric IDs
- IDs are mapped to dense vectors (embeddings)



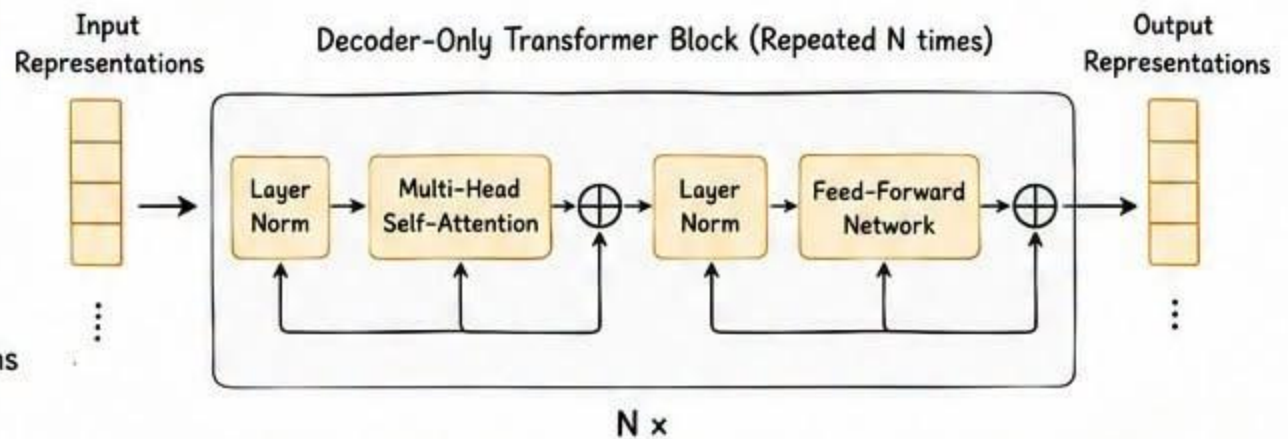
2. EMBEDDING LAYER

- Each token ID is converted into a dense vector
- These vectors provide learned representations of tokens
- Positional information is incorporated.



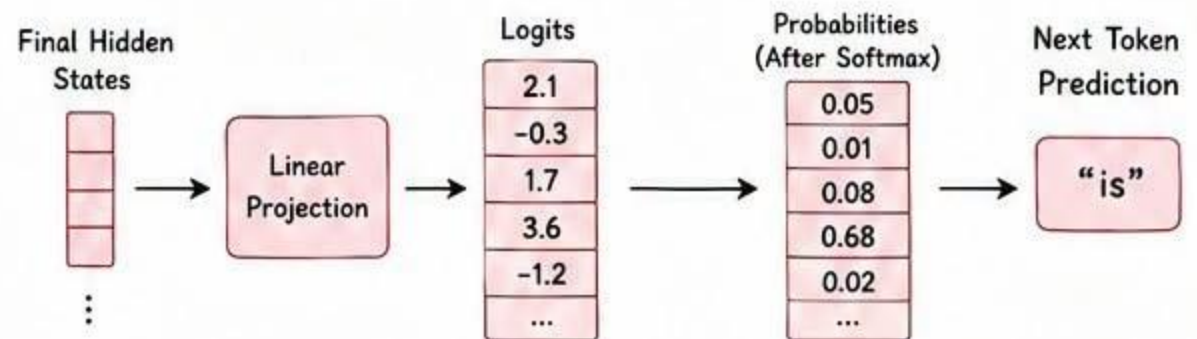
3. DECODER-ONLY TRANSFORMER BLOCKS (STACKED LAYERS)

- Many generative LLMs use a decoder-only Transformer.
- Each block has:
 - Multi-Head Self-Attention
 - Feed-Forward Network
 - Normalization & Residual Connections
- Many such blocks are stacked (e.g., 24, 32, 48, 70+)



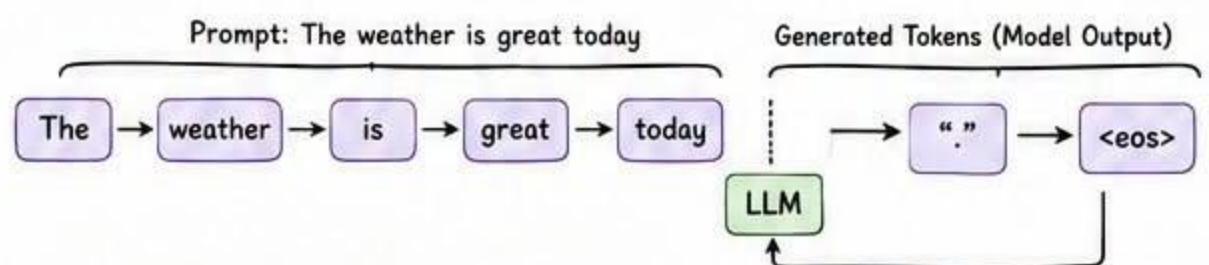
4. OUTPUT LAYER (LM HEAD)

- The hidden state at each position is projected to vocabulary logits.
- Produces logits (scores) for each token in vocabulary
- Softmax converts logits to probabilities



5. TEXT GENERATION

- The predicted token is appended to the input.
- The process repeats autoregressively to generate more tokens.
- Continues until a stopping condition is reached (e.g., <eos>).



Key Takeaway

LLMs use Transformer architectures to model context and generate text one token at a time.

- Input Text → Tokens → Embeddings
- Embeddings + Positional Information → Transformer Blocks
- Output Layer → Next Token → Repeat

Large Language Models (LLMs) in AI

Key Concepts • Architecture • Training • Examples • Applications

1. What are LLMs?

- Large Language Models (LLMs) are AI models trained on massive amounts of text data.
- They learn patterns in language to understand and generate human-like text.
- Built using deep learning, mainly transformer architecture.
- Can perform a wide range of tasks like answering questions, summarizing, coding and more.

“LLMs learn from text to generate meaningful and helpful responses.”

2. Transformer Architecture

- LLMs are based on the Transformer architecture.
- Uses self-attention to understand the context of words in a sentence.
- Processes training sequences in parallel, making training more efficient.

Self-Attention (Simplified):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Query K = Key V = Value
 d_k = dimension of keys

Example:

Input: “I love AI”
Model understands the relationship between words like “I” and “AI”.

3. Tokenization

- Text is split into smaller units called tokens (words, subwords, characters, or byte-level units).
- Each token is converted into a numerical representation called an embedding.

Example:

Text: “I love AI”
Tokens: [“I”, “love”, “AI”]
Token IDs: [101, 2034, 5678]
(IDs are just examples)

Tokenization methods:

- BPE (Byte Pair Encoding)
- WordPiece
- SentencePiece

4. Training LLMs

- Trained on large and diverse datasets, including books, websites, articles, code, and other text sources.
- Learns to predict the next token in a sequence (next-token prediction).
- Pretraining commonly uses self-supervised learning, where labels are automatically derived from the text.

Training Objective (Next-Token Prediction):

$$\mathcal{L} = - \sum_{t=1}^n \log P(x_t | x_1, \dots, x_{t-1})$$

Example:

Input: “The sky is”
Model predicts: “blue”

5. Inference (Text Generation)

- After training, the model can generate new text based on a prompt.
- Uses the learned patterns to predict the next most likely token.
- Different decoding strategies control the output.

Common Decoding Strategies:

- Greedy Search (pick most likely token)
- Beam Search (keep top candidates)
- Sampling (pick randomly based on probabilities)
- Top-k / Top-p (nucleus sampling)

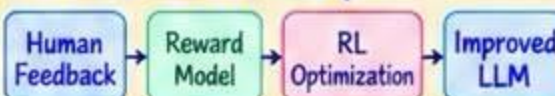
Example:

Prompt: “The future of AI is”
Output: “bright and full of opportunities.”

6. Fine-tuning & Alignment

- Fine-tuning adapts a pre-trained LLM to a specific task or domain (e.g., medical, legal, coding).
- Often uses a smaller task-specific dataset, which may be labeled or unlabeled depending on the objective.
- Alignment techniques (like RLHF) make the model safer, more helpful and follow human instructions.

RLHF (Reinforcement Learning from Human Feedback):



Example:

A general LLM can be fine-tuned to become a medical assistant, legal advisor or coding tutor.

7. Common LLM Families

- Many LLMs are available, each with different strengths.

Model	Organization	Key Features
GPT	OpenAI	Strong general capabilities
Claude	Anthropic	Safe and helpful responses
Gemini	Google	Multimodal (text, images, etc.)
Llama	Meta	Open-source and customizable
Mistral	Mistral AI	Efficient and high performance

“Different models have different strengths — choose the right tool for your need.”

8. Applications of LLMs

- Chatbots & Virtual Assistants (e.g., customer support)
- Content Generation (e.g., blogs, emails, summaries)
- Code Generation & Debugging (e.g., AI coding assistants such as GitHub Copilot)
- Search & Research (e.g., summarize information and answer questions with supporting tools)
- Education & Tutoring (e.g., explain concepts, learning support)
- Automation & Productivity (e.g., writing, documentation, analysis)
- Creative Tasks (e.g., stories, ideas, brainstorming)

9. Key Concepts to Remember

- ✓ Tokens and tokenization
- ✓ Embeddings
- ✓ Transformer and self-attention
- ✓ Next-token prediction
- ✓ Fine-tuning and RLHF
- ✓ Context window
- ✓ Prompt engineering
- ✓ Evaluation metrics

10. Challenges & Limitations

- ⚠ Can generate incorrect or misleading information (hallucinations).
- ⚠ Limited knowledge (depends on training data and available external tools).
- ⚠ Biased or unfair outputs.
- ⚠ May lack real-time information without external tools or browsing.
- ⚠ Can be computationally expensive.

11. Learning Path

- 1 Understand how LLMs work (transformers).
- 2 Learn about tokenization and embeddings.
- 3 Explore prompt engineering.
- 4 Try using LLM APIs (e.g., OpenAI, Claude).
- 5 Experiment with fine-tuning.
- 6 Learn about model alignment and safety.
- 7 Build real-world projects.

12. Key Takeaways

- ✓ LLMs are powerful tools for understanding and generating language.
- ✓ They are built using the Transformer architecture.
- ✓ Prompting, fine-tuning and alignment help tailor them for specific needs.
- ✓ They have many real-world applications across industries.
- ✓ Use them responsibly and be aware of their limitations.

Transformers

Cheat Sheet

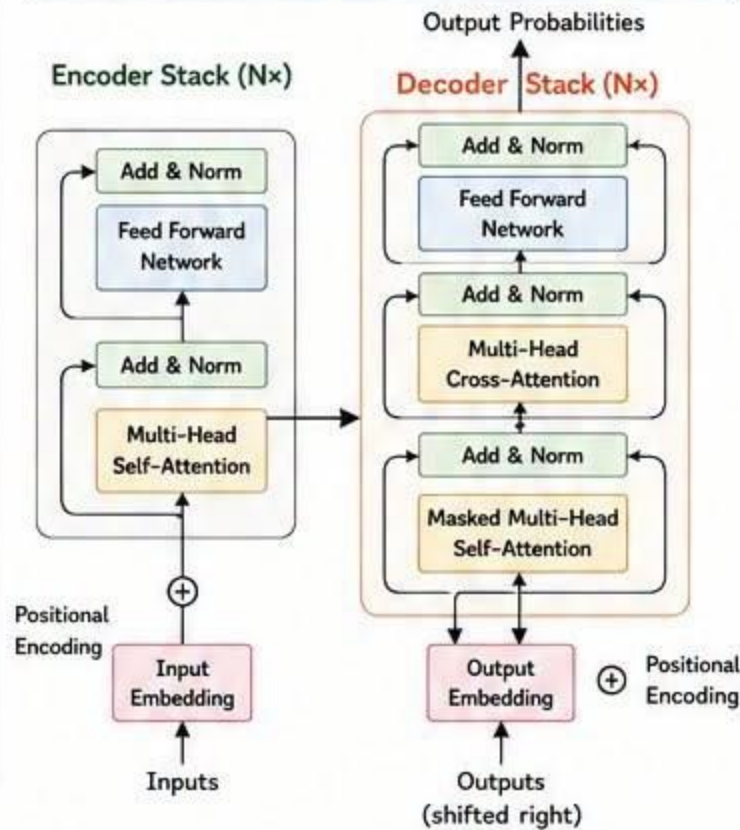
1. What are Transformers?



Transformers are a type of neural network architecture introduced in the paper "Attention Is All You Need" (2017). They rely on self-attention instead of recurrence or convolution.

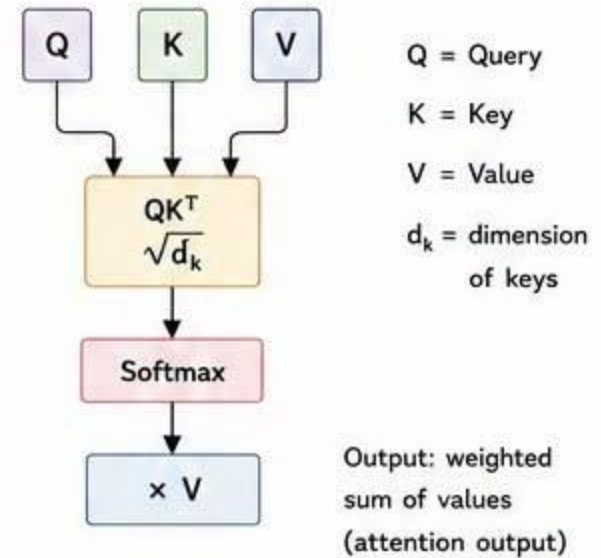
- ✓ Highly parallelizable during training
- ✓ Capture long-range dependencies
- ✓ Widely used in NLP and beyond

2. High-Level Architecture



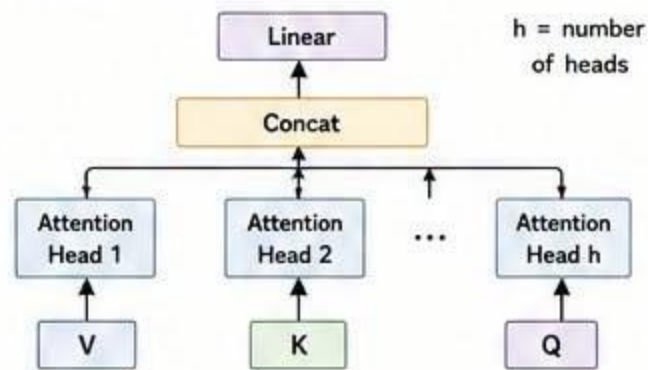
3. Self-Attention

Compute attention of a token with all tokens in the sequence.



4. Multi-Head Attention

Run multiple attention heads in parallel and concatenate their outputs.



5. Position-wise Feed Forward Network

Applied independently to each position.

Example: Linear $\rightarrow$ ReLU $\rightarrow$ Linear

Original Transformer:

$$d_{ff} = 4 \times d_{model}$$

(e.g., 2048 when $d_{model} = 512$)

6. Positional Encoding

Since transformers have no recurrence, we add positional information.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos = position, i = dimension index

0	1	2	2	3	4	...
---	---	---	---	---	---	-----

Added to input embeddings

7. Masked Multi-Head Attention (Decoder)

Prevents looking at future tokens.

Look-ahead mask (example):

Tokens $\rightarrow$	T1	T2	T3	T4
T1	✓	✗	✗	✗
T2	✓	✓	✗	✗
T3	✓	✓	✓	✗
T4	✓	✓	✓	✓

✓ allowed ✗ not allowed

8. Complexity

Transformers have better parallelization but higher memory cost.

Model Type	Complexity (with respect to n)
RNN	$O(n)$ (sequential operations)
Self-Attention (Transformers)	$O(n^2)$ (w.r.t. n)

n = sequence length

9. Common Use Cases

- 🌐 Machine Translation
- 📄 Text Summarization
- ❓ Question Answering
- 🏷️ Text Classification
- 😊 Sentiment Analysis
- </> Code Generation
- 🔊 Speech & Vision Tasks

10. Hugging Face Quick Start

```
from transformers import pipeline
# Text generation
pipe = pipeline("text-generation",
                model="gpt2")
result = pipe("Transformers are",
              max_length=30)
print(result[0]['generated_text'])
```

Hugging Face provides easy-to-use Transformer APIs.

11. Load a Pretrained Model (BERT)

```
from transformers import AutoTokenizer,
                        AutoModel

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(
    model_name)
model = AutoModel.from_pretrained(model_name)

inputs = tokenizer("Hello, world!",
                  return_tensors="pt")
outputs = model(**inputs) # model outputs
```

BERT is an encoder-only model. With task-specific heads, it can be used for classification, QA, and more.

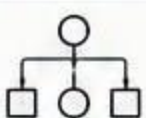
12. Fine-Tuning (Outline)

- 1 Load dataset
- 2 Tokenize the data
- 3 Create DataLoader
- 4 Load pretrained model
- 5 Define loss & optimizer
- 6 Train & evaluate model

Use Trainer API in Hugging Face for simpler fine-tuning.

13. Key Hyperparameters

- d_{model} : embedding dimension (e.g., 512, 768, 1024)
- num_heads : number of attention heads (e.g., 8, 12, 16)
- num_layers : number of encoder/decoder layers (e.g., 6, 12, 24)
- d_{ff} : feed-forward hidden size (usually $4 \times d_{model}$)
- $dropout$: regularization (e.g., 0.1)
- max_seq_length : max tokens



Transformers = Attention + Parallelization + Feed-Forward Networks
Powering the most advanced AI models today.

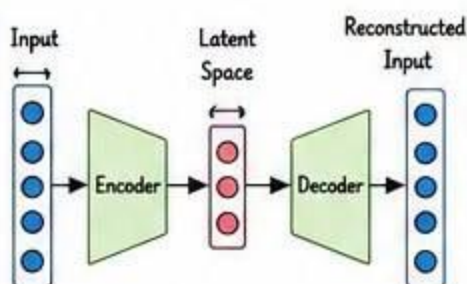
Remember:
Self-Attention is the core idea that makes Transformers powerful.

Generative AI Models

A quick guide to the most popular generative AI models

1. Autoencoder

Learns a compressed representation of the input and reconstructs it back as the original.



Key Features

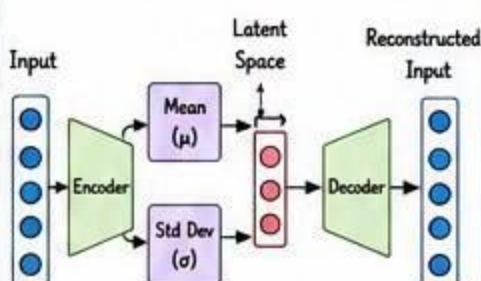
- Unsupervised learning
- Learns compact representations
- Useful for dimensionality reduction
- Can be used for anomaly detection

Best Used For

- Dimensionality reduction
- Anomaly detection
- Data denoising
- Feature learning

2. Variational Autoencoder (VAE)

A probabilistic version of autoencoder that learns a distribution in the latent space for generating new samples.



Key Features

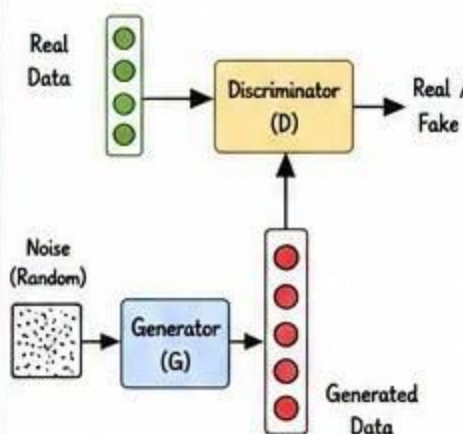
- Probabilistic latent space
- Can generate new, similar data
- Continuous latent space suitable for sampling
- Great for data generation

Best Used For

- Data generation
- Image generation
- Anomaly detection
- Latent space exploration

3. Generative Adversarial Network (GAN)

Uses a generator and a discriminator trained together so the generator learns to produce realistic data.



Key Features

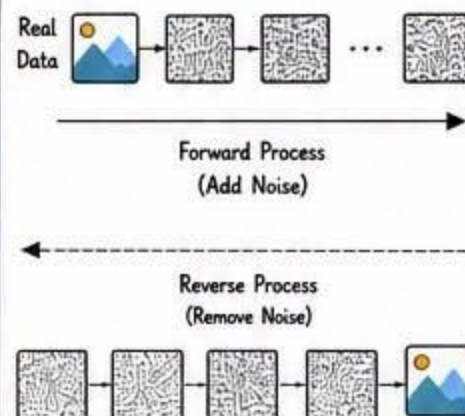
- Generates highly realistic data
- Learns complex data distributions
- Unsupervised training
- Two networks trained together

Best Used For

- Image generation
- Data synthesis
- Style transfer
- Super-resolution

4. Diffusion Models

Learns to remove noise step by step to generate new, high-quality data from random noise.



Key Features

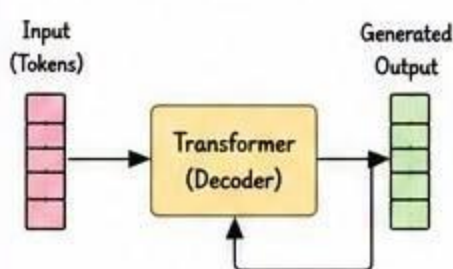
- Produces high-quality results
- Very stable training
- Works well with complex data
- State-of-the-art generation quality

Best Used For

- Image generation
- Art generation
- Text-to-image generation
- High-fidelity synthesis

5. Decoder-Only Transformer (Autoregressive Models)

Generates text or tokens one at a time using only previous context.



Key Features

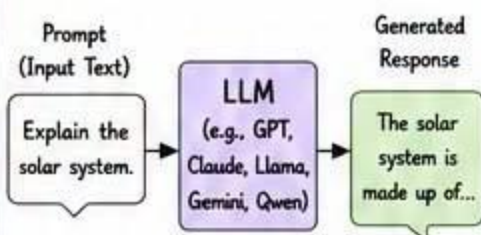
- Captures long-range dependencies
- Generates high-quality sequences
- Scalable to very large models
- Excels in text and code generation

Best Used For

- Text generation
- Code generation
- Summarization
- Translation
- Chatbots & assistants

6. Large Language Models (Examples of Autoregressive Transformers)

Pre-trained on massive text data to understand, generate, and reason about natural language.



Key Features

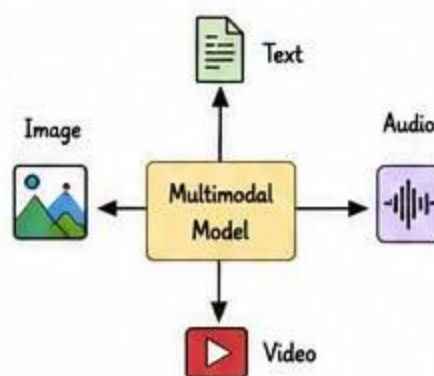
- Understands and generates text
- Pre-trained on huge datasets
- Can be fine-tuned or used via APIs
- Supports a wide range of tasks and domains

Best Used For

- Chatbots & assistants
- Content creation
- Q&A and reasoning
- Coding assistance

7. Multimodal Models

Understand and generate multiple data types such as text, images, audio, and video.



Key Features

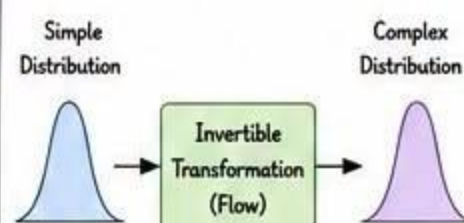
- Works with multiple data types
- Better context understanding
- Enables rich and interactive output
- Useful for real-world applications

Best Used For

- Image captioning
- Video understanding
- Audio generation
- Multimodal assistants

8. Normalizing Flow Models (Flow-Based Models)

Uses invertible transformations to map simple distributions to complex data distributions.



Key Features

- Exact likelihood calculation
- Reversible and invertible
- Stable and efficient training
- Good for high-dimensional data

Best Used For

- Density estimation
- Data generation
- Anomaly detection
- Scientific applications

What Should You Learn Before Starting Generative AI?

Your path depends on your goal!

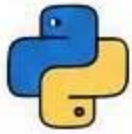
There isn't a single fixed path. Here are two common learning paths depending on what you want to achieve.

Path 1: Build Generative AI Applications

(Practical / Faster Path)

Good if you want to start building projects early!

1 Python Basics



- Syntax and core concepts
- Work with data (e.g. pandas)
- Basic scripting

2 APIs & Development Basics



- How APIs work
- Make API calls (e.g. OpenAI)
- Basic web app development
- Use Git for version control

3 LLM Fundamentals



- What LLMs are
- Capabilities and limitations
- Different models (GPT, Claude, etc.)
- How to interact with them

4 Prompt Engineering



- How to write effective prompts
- Prompting techniques (e.g. few-shot)
- Evaluate and improve outputs

5 RAG, Tools & AI Agents



- Build RAG applications
- Use tools and APIs
- Explore AI agents (next step)
- Work on real-world projects

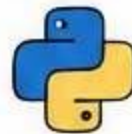
You can start building useful projects after the first few steps!

Path 2: Understand Generative AI Deeply

(In-Depth / Research-Oriented Path)

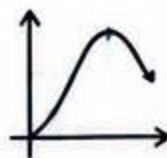
Good if you want to understand how models work under the hood!

1 Python Programming



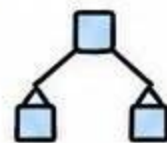
- Strong Python fundamentals
- Data manipulation and visualization
- Useful libraries (e.g. numpy, pandas)

2 Math Foundations



- Linear algebra
- Probability and statistics
- Calculus (basics)
- Optimization (basic idea)

3 Machine Learning



- Core ML concepts
- Model training and evaluation
- Classical algorithms
- Hands-on practice

4 Deep Learning



- Neural networks
- Training basics (forward/backward pass)
- CNNs, RNNs (overview)
- Frameworks (e.g. PyTorch)

5 Transformers & LLMs



- Transformer architecture
- Attention mechanism
- Pre-training and post-training (e.g. SFT, RLHF)
- LLM training and inference concepts

You Can Skip (For Now)

← Not essential to get started!

- ☐ Advanced mathematics (you can skip initially if your goal is to build applications)
- ☐ Every deep learning architecture
- ☐ Reading all research papers
- ☐ Learning every tool or framework
- ☐ Becoming an expert before building

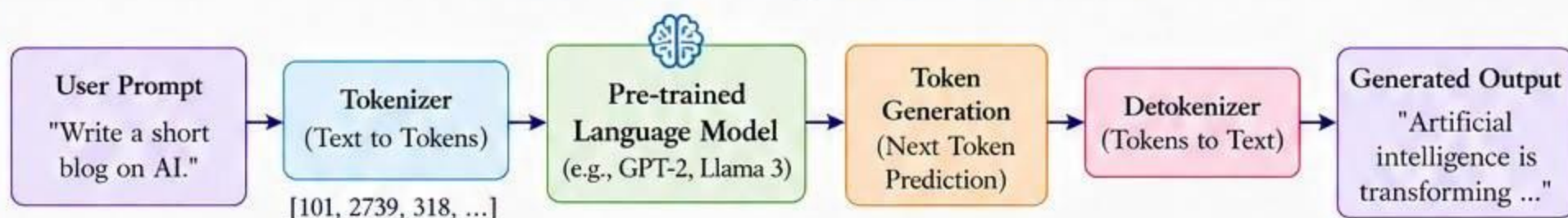
Keep in Mind

- ☒ You don't need to learn everything before starting.
- ☒ Start with your goal and learn as needed.
- ☒ Build small projects early.
- ☒ Focus on concepts, not just tools.
- ☒ Practice consistently.
- ☒ The field is evolving - keep learning!

Start where you are.
Use what you have.
Build something.
You'll learn the rest
along the way. 😊

Generative AI Project

Text Generation using a Pre-trained LLM



Project Goal:

Generate coherent, human-like text using a pre-trained large language model (LLM).

How it works:

1. **Input Prompt:** You provide a text prompt.
2. **Tokenization:** The text is converted into tokens (numeric representation).
3. **Pre-trained Model:** A large language model predicts the next token based on context.
4. **Decoding:** The predicted tokens are converted back into readable text.

Model Options:

- OpenAI models (via API)
- Meta Llama (via Hugging Face)
- Mistral, Gemma, Phi, etc.

Key Concepts:

- Transformer architecture (self-attention)
- Pre-trained on large text data
- Generates text token by token
- Can use probabilistic sampling (e.g., temperature, top-k)

Evaluation Metrics (for text generation):

- Perplexity (lower is better)
- BLEU / ROUGE (when reference text is available)
- Human evaluation (fluency, relevance, coherence)

Applications:

- Chatbots & virtual assistants
- Content creation (blogs, emails, scripts)
- Code generation
- Summarization
- Creative writing

Generative AI can create new content (text, code, images, audio) by learning patterns from large amounts of data.

Generative AI Project: Python Code

```
# Install dependencies (comment if needed)
# !pip install transformers torch

from transformers import AutoTokenizer, AutoModelForCausalLM

def load_model(model_name="gpt2"):
    """Load tokenizer and model."""
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)
    return tokenizer, model

def generate_text(prompt, model, tokenizer, max_new_tokens=100,
                  temperature=0.8, top_k=50):
    """Generate text using the model."""
    # Tokenize input
    inputs = tokenizer(prompt, return_tensors="pt")

    # Generate output
    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        top_k=top_k,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id
    )

    # Decode tokens to text
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return generated_text

# Main
if __name__ == "__main__":
    model_name = "gpt2" # Try other models: "meta-llama/Meta-Llama-3-8B"
    tokenizer, model = load_model(model_name)
    prompt = "Write a short blog on the benefits of learning AI."
    result = generate_text(prompt, model, tokenizer)

    print("\nGenerated Text:\n")
    print(result)
```

Example

```
prompt = "Write a short blog on !"
output = generate_text(prompt, model, tokenizer)
print(output)
```



Sample Output (results may vary)

Generated Text:

Learning AI opens up a world of opportunities. It helps you solve real-world problems, automate repetitive tasks, and build innovative solutions. With AI skills, you can explore exciting career paths in data science, machine learning, and software engineering. Most importantly, learning AI empowers you to stay relevant in a rapidly evolving, technology-driven world.

You can also try:

- Change the model: "meta-llama/Meta-Llama-3-8B", "mistralai/Mistral-7B-Instruct", "google/gemma-7b"
- Adjust parameters: temperature, top_k, max_new_tokens
- Use a conversational prompt (e.g., for chatbot)
- Build a simple web app using Streamlit

Generative AI Libraries & Frameworks to Learn (2026)

1. PyTorch

Deep learning library for building generative models.



- Flexible & dynamic
- Great for research
- Strong GPU support

```
import torch
from torch import nn
```

- Best for: Building custom models
- Example: GANs, VAEs, Diffusion Models

2. Hugging Face Transformers

State-of-the-art models for NLP, multimodal & more.



- 1000s of pretrained models
- Easy to use
- Active community

```
from transformers import pipeline
pipe = pipeline("text-generation")
```

- Best for: LLMs, text generation, summarization, translation
- Example: GPT, BERT, T5, LLaMA

3. LangChain

Build LLM applications with chains, memory & tools.



- Chains & agents
- Memory management
- Integrates with tools & APIs

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
agent = create_agent(model=model, tools=[])
```

- Best for: LLM apps, chatbots, automation
- Example: Q&A bots, RAG apps, Agents

4. Diffusers

Create diffusion models for images, video & audio.



- Stable Diffusion
- High performance
- Supports pipelines

```
from diffusers import
StableDiffusionPipeline
```

- Best for: Image & video generation
- Example: Text-to-image, Inpainting, Video generation

5. OpenAI Python Library

Official library to build apps with OpenAI models.



- Easy API access
- Chat, Embeddings, Images & more
- Reliable & updated

```
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")
```

- Best for: GPT apps, embeddings, image generation
- Example: Chatbots, content generation

6. LlamaIndex

Connect LLMs with your data using indexes & retrieval.



- Data connectors
- Indexing & retrieval
- Works great with RAG

```
from llama_index.core import
SimpleDirectoryReader
```

- Best for: RAG, knowledge bases
- Example: Docs Q&A, research assistants

7. Cohere

Advanced NLP models & tools for generative AI.



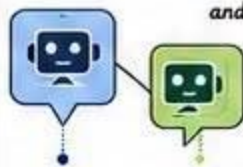
- Text generation
- Embeddings
- Rerank & classify

```
import cohere
co = cohere.ClientV2(api_key="YOUR_API_KEY")
```

- Best for: Enterprise NLP apps
- Example: Summarization, Search, Classify

8. AutoGen (Maintenance Mode)

Build multi-agent conversations and autonomous AI apps.



- Multi-agent workflows
- Agent collaboration
- Customizable

Legacy API example:

```
from autogen import AssistantAgent,
UserProxyAgent
```

Note: AutoGen is in maintenance mode.
For new projects, consider Microsoft Agent Framework.

- Best for: Multi-agent AI systems
- Example: Autonomous assistants, Task agents

9. Gradio

Create beautiful UI for your AI/ML models in minutes.



- Easy & fast
- Share demos
- No frontend skills needed

```
import gradio as gr
gr.Interface(...).launch()
```

- Best for: Model demos & prototypes
- Example: Chatbots, Image generators

10. LangSmith

Debug, test & monitor LLM apps built with LangChain.



- Tracing & Logging
- Evaluate prompts
- Improve performance

```
from langsmith import Client
client = Client()
```

- Best for: LLM app observability
- Example: Debug chains, track performance

11. bitsandbytes

Optimize LLMs with quantization for faster & cheaper inference.



```
from transformers import AutoModelForCausalLM,
BitsAndBytesConfig

config = BitsAndBytesConfig(load_in_8bit=True)

model = AutoModelForCausalLM.from_pretrained(
    "MODEL_NAME",
    quantization_config=config,
    device_map="auto"
)
```

- Best for: Running large models efficiently
- Example: LLaMA, Falcon, Mistral

12. Transformers.js

Run transformer models in the browser with JavaScript.



- Browser-based
- No backend needed
- Web-friendly AI

```
import { pipeline } from
"@huggingface/transformers"
```

- Best for: Web apps with AI
- Example: Text generation, Translation

AI FRAMEWORKS & TOOLS IN 2026

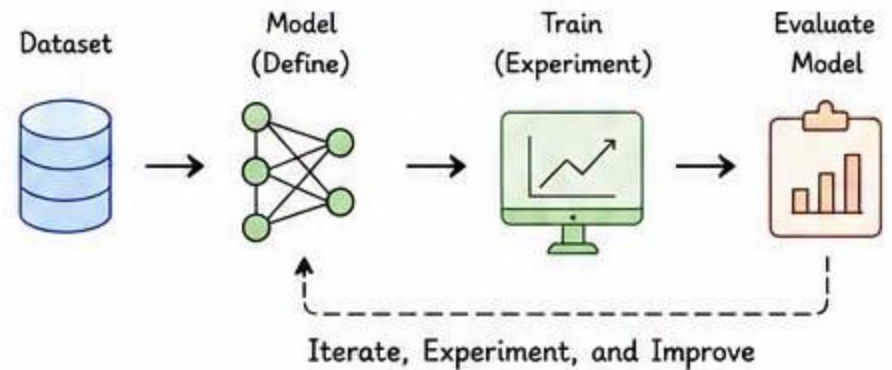
Modern AI frameworks and tools provide the libraries and engines to build, train, and deploy AI systems efficiently.

1. PYTORCH

- Originally developed by Meta AI; open-source
- Eager execution & dynamic graphs
- Excellent for research & training
- Strong ecosystem (TorchVision, TorchAudio, Hugging Face, etc.)
- Widely used for LLMs & Generative AI



Research & Training Workflow

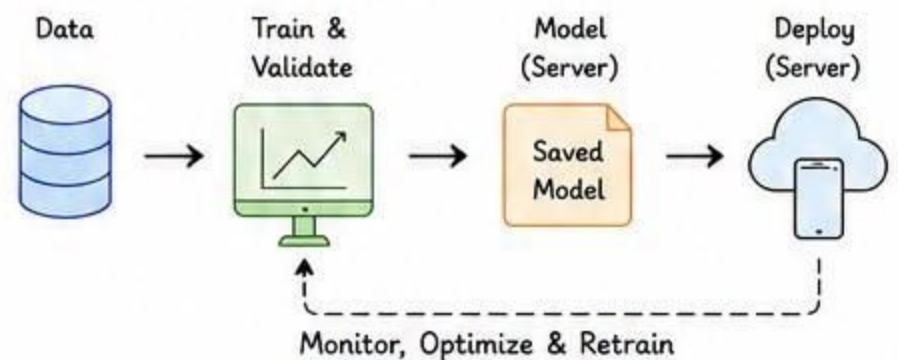


2. TENSORFLOW

- Developed by Google
- Flexible and production-ready
- Great for both research and production
- TensorFlow with Keras 3
- Strong deployment ecosystem (TF Serving, LiteRT, TF.js)



Production ML & Deployment Workflow

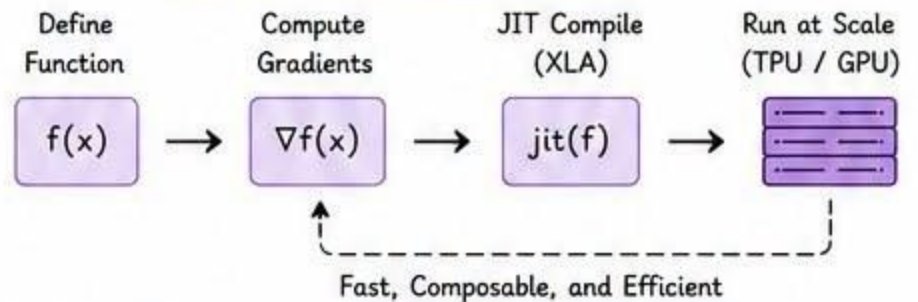


3. JAX

- Developed by Google
- High-performance numerical computing with XLA compilation
- Automatic differentiation (`grad()`)
- Ideal for research & large-scale models
- Great for accelerators (TPU, GPU)



High-Performance Research Workflow

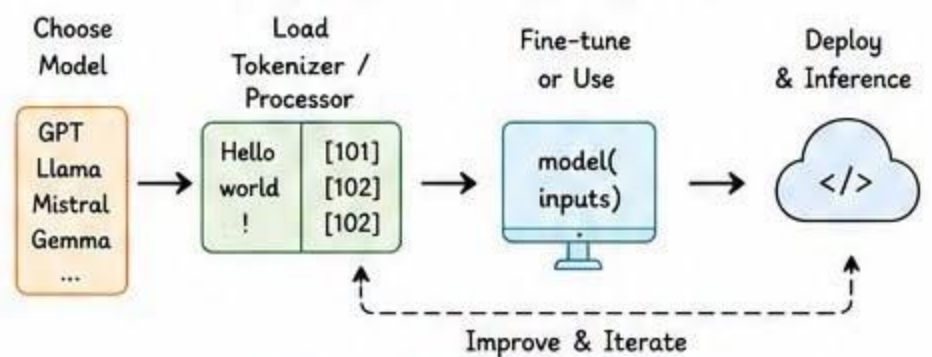


4. HUGGING FACE TRANSFORMERS

- Developed by Hugging Face
- Leading library for NLP, LLMs, CV, and Multimodal AI
- Thousands of pretrained models
- Easy to use and integrate
- Fast adoption & active community



Pretrained Models & Multimodal AI Workflow

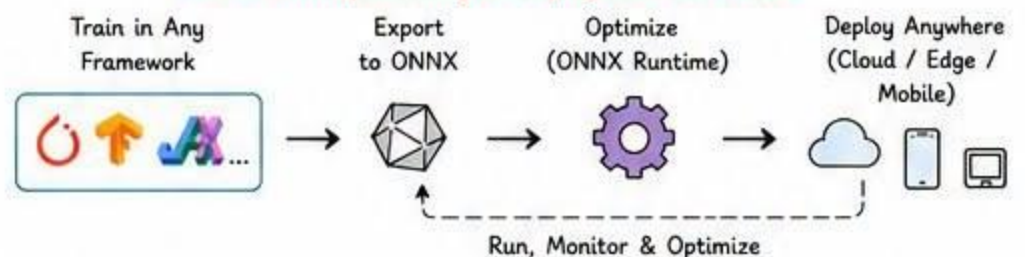


5. ONNX (MODEL INTEROPERABILITY)

- Open Neural Network Exchange
- Framework-agnostic model format
- Interoperability across frameworks and platforms
- Common format for deployment workflows
- Great for production & edge



Model Interoperability & Deployment Workflow

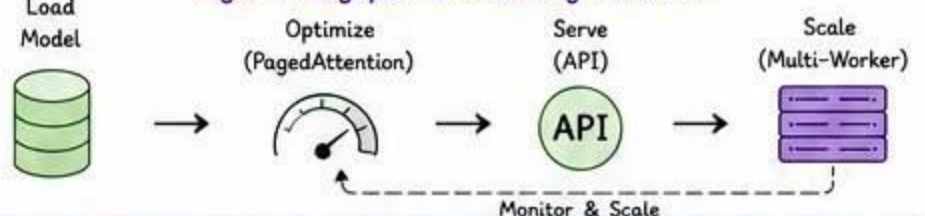


6. vLLM (LLM INFERENCE & SERVING)

- High-throughput LLM inference engine
- PagedAttention for efficient memory usage
- Supports an OpenAI-compatible API
- Ideal for serving LLMs at scale



High-Throughput LLM Serving Workflow



Key Takeaway

AI frameworks and tools in 2026 empower you to build, train, fine-tune, and deploy intelligent systems efficiently. Choose the right tool based on your goals—research, production, scalability, or LLM serving.

- PyTorch → Research & Flexibility
- TensorFlow → Production & Deployment
- JAX → High-Performance Research (grad, JIT, XLA)
- Hugging Face → Pretrained Models & Multimodal AI
- ONNX → Model Interoperability & Deployment
- vLLM → High-Throughput LLM Inference & Serving